Artificial Intelligence Neural Networks

Basanta Joshi, PhD

Assistant Professor, Department of Electronics and Computer Engineering,

Pulchowk Campus https://pcampus.edu.np/

Deputy Director, Center for Applied Research & Development(CARD)

https://card.ioe.edu.np/

Member, Laboratory for ICT Research and Development (LICT) http://lict.ioe.edu.np/

Institute of Engineering, Tribhuvan University

basanta@ioe.edu.np

http://www.basantajoshi.com.np

https://scholar.google.com/citations?user=iocLiGcAAAAJ

https://www.researchgate.net/profile/Basanta_Joshi2



notes.arpanadhikari7.com

Introduction to Neural Nets

notes.arpanadhikari7.com.np

Introduction

Available on notes.arpanadhikari7.com.np

to Neural Nets

The Human Brain

notes.arpanadhikari7.com.np

- Computers and the Brain: A Contrast
  - Arithmetic: 1 brain = 1/10 pocket calculator
  - Vision: 1 brain = 1000 super computers
  - Memory of arbitrary details: computer wins
  - Memory of real-world facts: brain wins

Available on notes.arpanadhikari7.com.np

- A computer must be programmed explicitly
- The brain can learn by experiencing the world

Other Comparisons

notes.arpanadhikari7.com.np

ITEM	COMPUTER	BRAIN
Complexity	ordered structure serial processor	Processor Speed 10,000,000 operations per second

Available on notes.arpanadhikari7.com.np

10^10 neuron processors 10^4 connections 100	operations per second	
Computational Power	one operation at a time 1 or 2 inputs	millions of operations at a time thousands of inputs

notes.arpanadhikari7.com.np

Computer Operations

Traditionally computers execute a sequence of instructions to accomplish a set task

This is a powerful technique if you know the ‘algorithm’

Available on notes.arpanadhikari7.com.np

It’s not very useful if you don’t !!

There are many interesting tasks where the algorithm is either unknown or unclear

- Recognizing handwriting - Pattern recognition
- Playing table tennis - Interacting with the environment
- Balancing activities - Optimization

notes.arpanadhikari7.com.np

Definition

Available on notes.arpanadhikari7.com.np

- “. . . Neural nets are basically mathematical models of information processing . . .”

- “. . . (neural nets) refer to machines that have a structure that, at some level, reflects what is known of the structure of the brain . . .”

notes.arpanadhikari7.com.np

- “A neural network is a massively parallel distributed processor . . . “

Available on notes.arpanadhikari7.com.np

Neural Net Concept

- Artificial Neural Systems are called:
  - neurocomputers
  - neural networks
  - parallel distributed processors
  - connectionists systems

notes.arpanadhikari7.com.np

- Basic Philosophy
  - large number of simple “neuron-like” processors which execute global or distributed computation

Processing Concept

notes.arpanadhikari7.com.np

INPUT
(Type)

(Procedural) Data Input

1, 2, 3, etc.
Conventional

Processing

Procedures applied

OUTPUT

to data

Data Output

(Logical) Expert/Knowledge-Based System

Facts

Knowledge (Rules)

Decision

Recommend

(A is True) (Statistical)

Inference Engine &

Neural Networks

Patterns Pattern Network Algorithm Identifies

Patterns

recognized

notes.arpanadhikari7.com.np

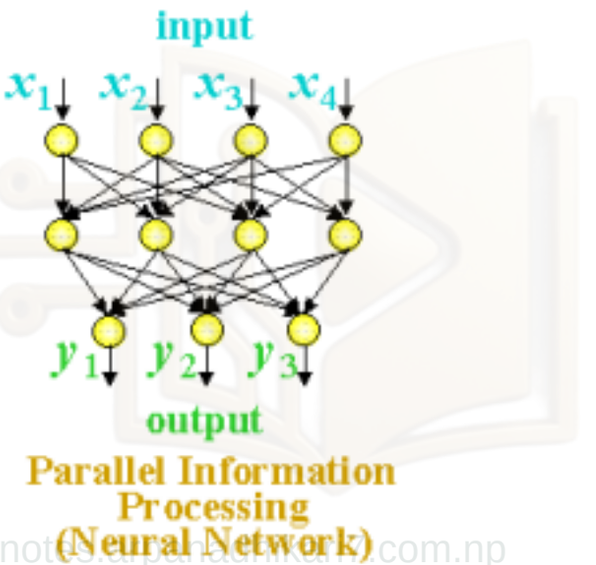
Features of Neural Nets

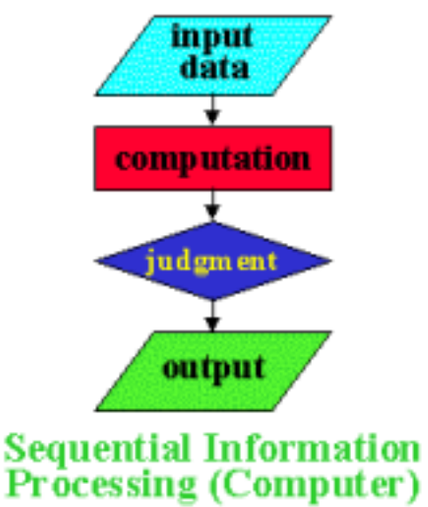
Available on notes.arpanadhikari7.com.np

Neural Net Characteristics 1

- Neural nets have many unique features:

Massive Parallel Processing:



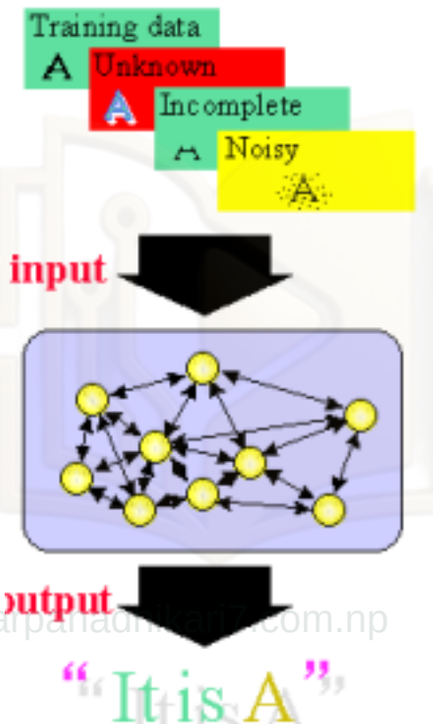


Neural Nets update in parallelComputers calculate in sequence

Available on notes.arpanadhikari7.com.np

Neural Net Characteristics 2

- Neural nets can learn:



Once a neural net is trained, it can handle unknown, incomplete or noisy data

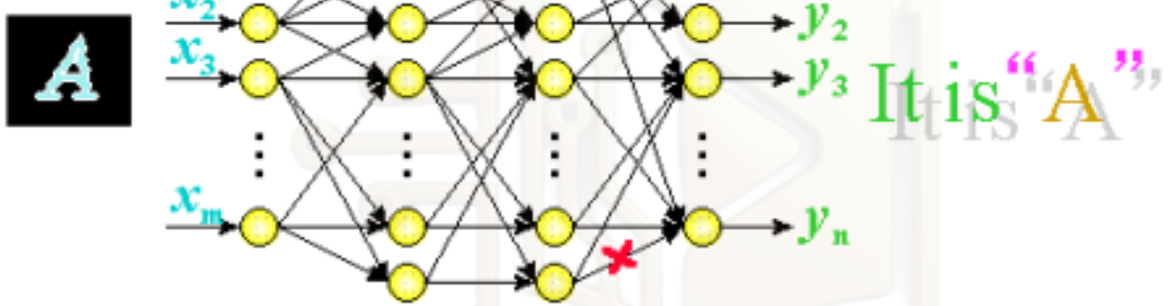
notes.arpanadhikari7.com.np

Neural Net Characteristics 3

- Unlike computer memory, information is distributed over the entire network, so a failure in one part of the network does not prevent the system for producing the correct output.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Learning in
Neural Nets

notes.arpanadhikari7.com.np

Learning in

Available on notes.arpanadhikari7.com.np

Neural Nets

notes.arpanadhikari7.com.np

Slide | 13 By: Basanta Joshi, PhD

Learning

- One of the powerful features of neural networks is learning.

Available on notes.arpanadhikari7.com.np

- Learning in neural networks is carried out by adjusting the connection weights among neurons.

- It is similar to a biological nervous system in which learning is carried out by changing synapses connection strengths, among cells.

- Learning may be classified into 2 categories:
  (1) Supervised Learning
  (2) Unsupervised Learning

notes.arpanadhikari7.com.np

Supervised Learning

Available on notes.arpanadhikari7.com.np

- In supervised learning, the network is presented with inputs together with the target (teacher signal) outputs.

- Then, the neural network tries to produce an output as close as possible to the target signal by adjusting the values of internal weights.

notes.arpanadhikari7.com.np

- The most common supervised learning method is the **“error correction method”**

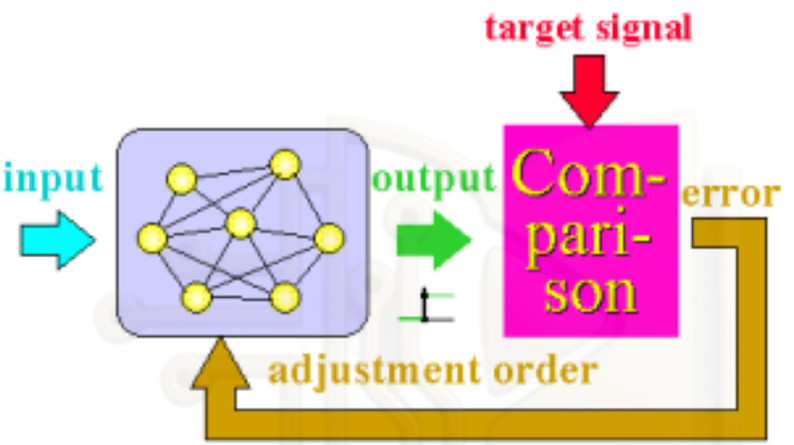
Available on notes.arpanadhikari7.com.np

Error Correction Method

- Error correction method is used for networks which their neurons have discrete output functions.

- Neural networks are trained with this method in order to reduce the error (difference between the network's output and the desired output) to zero.

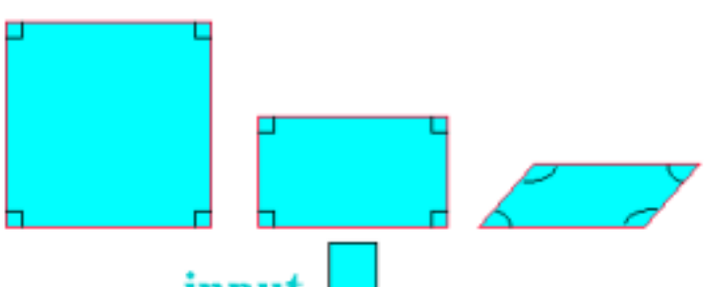
notes.arpanadhikari7.com.np



Unsupervised Learning

Visit arpanadhikari7.com.np

• In unsupervised learning, there is



Available on notes.arpanadhikari7.com.np

no teacher (target signal) from outside and the network adjusts its weights in response to only the input patterns.

- A typical example of unsupervised learning is Hebbian learning.
- clustering
- hidden Markov models,
- self-organizing map (SOM) etc.

notes.arpanadhikari7.com.np

Perceptron

Available on notes.arpanadhikari7.com.np

- In 1958, **Frank Rosenblatt** introduced a training algorithm that provided the first procedure for training a simple ANN: a **perceptron**.
- The perceptron is the simplest form of a neural network. It consists of a single neuron with *adjustable* synaptic weights and a *hard limiter*.
- The operation of Rosenblatt's perceptron is based on the **McCulloch and Pitts neuron model**. The model consists of a linear combiner followed by a hard limiter.
- The weighted sum of the inputs is applied to the hard limiter, which produces an output equal to +1 if its input is positive and −1 if it is negative.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Single-layer two-input perceptron



notes.arpanadhikari7.com.np

$\sum Y$

Threshold

x_2

w_2

θ

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

McCulloch/Pitts Neuron

McCulloch/Pitts Neuron

notes.arpanadhikari7.com.np

Common Features

- "weight" = strength of connection
- threshold = value of weighted input below which no response is produced
- signals may be:
  - real-valued, or
  - binary-valued:

Available on notes.arpanadhikari7.com.np

- “unipolar” {0, 1}
- “bipolar” {-1, 1}

McCulloch/Pitts Neuron

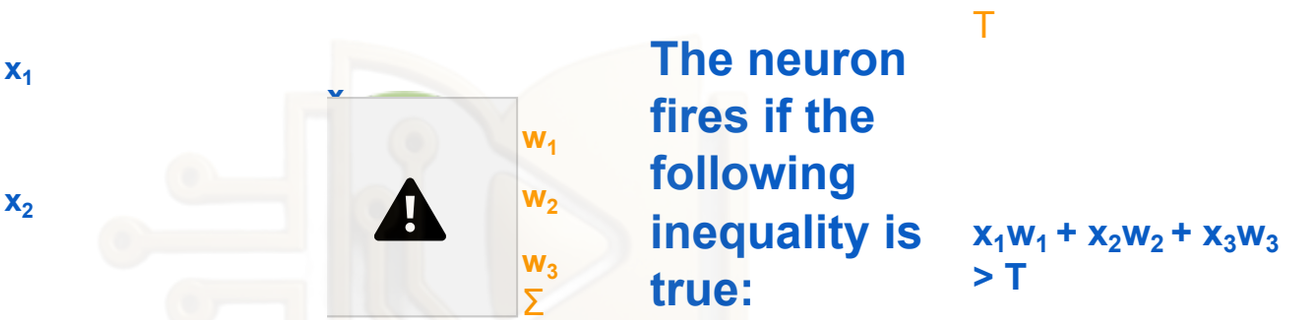
notes.arpanadhikari7.com.np

- One of the first neuron models to be implemented
- Its output is 1 (fired) or 0
- Each input is weighted with weights in the range -1 to

Available on notes.arpanadhikari7.com.np

+ 1

- It has a threshold value, T



Single Neuron Application

notes.arpanadhikari7.com.np

- **GOAL**: Construct an MCP (McCulloch/Pitts) neuron which will implement the function below:

Available on notes.arpanadhikari7.com.np

PROBLEM: find the threshold, T, and the weights w_1 and w_2 where:

F is 1 if $x_1w_1 + x_2w_2 > T$

x1	x2	F
0	0	0
0	1	1
1	0	1
1	1	1

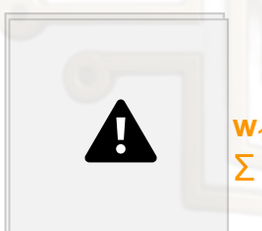
Each line of the function table places conditions on the unknown values:

What is this function?

$0 < w_2 > T$

$w_1 > T$

$w_1 + w_2 > T$



w_1 and w_2 = 0.7 T = 0.5 works

notes.arpanadhikari7.com.np

Perceptron

Available on notes.arpanadhikari7.com.np

- The aim of the perceptron is to classify inputs,
  - x1, x2, . . ., xn, into one of two classes, say
  - A1 and A2.
- In the case of an elementary perceptron, the n-dimensional space is divided by a hyperplane into two decision regions. The hyperplane is defined by the linearly separable function:

notes.arpanadhikari7.com.np

$$\sum_{i=0}^{n} x_1 - \theta =$$

Available on notes.arpanadhikari7.com.np

$_i w_i$

Perceptron

- How does the perceptron learn its classification tasks?
- This is done by making small adjustments in the weights to reduce the difference between the actual and desired outputs of the perceptron. The initial weights are randomly assigned, usually in the range [-0.5, 0.5], and then updated to obtain the output consistent with the training examples.
  - If at iteration p, the actual output is $Y(p)$ and the desired output is $Y_d(p)$, then the error is given by:

Available on notes.arpanadhikari7.com.np

$$e(p) = Y_d(p) - Y(p)$$

- Iteration p here refers to the pth training example presented to the perceptron.
- If the error, $e(p)$, is positive, we need to increase perceptron output $Y(p)$, but if it is negative, we need to decrease $Y(p)$.

The perceptron learning rule

notes.arpanadhikari7.com.np

$$w_i(p+1) = w_i(p) + \alpha \cdot x_i(p) \cdot e(p)$$

Available on notes.arpanadhikari7.com.np

where $p = 1, 2, 3, \ldots$

α is the learning rate, a positive constant less than unity.

- The perceptron learning rule was first proposed by Rosenblatt in 1960. Using this rule we can derive the perceptron training algorithm for classification tasks.

notes.arpanadhikari7.com.np

Perceptron's training algorithm

Available on notes.arpanadhikari7.com.np

Step 1: Initialisation

Set initial weights w_1, w_2,…, $w_n$and threshold θ to random numbers in the range [−0.5, 0.5].

If the error, $e(p)$, is positive, we need to increase perceptron output $Y(p)$, but if it is negative, we need to decrease $Y(p)$.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Perceptron's training algorithm

notes.arpanadhikari7.com.np

Step 2: Activation

Activate the perceptron by applying inputs $x_1(p)$, $x_2(p), \ldots, x_n(p)$ and desired output $Y_d(p)$.

Calculate the actual output at iteration $p = 1$

Available on notes.arpanadhikari7.com.np

$$= \sum_{i=1}^{n} - \theta \; Y\,p\; step\; x_i p\; w_i p\; (\;)\;(\;)\;(\;)$$

where n is the number of the perceptron inputs, and *step* is a step activation function.

notes.arpanadhikari7.com.np

Perceptron's training algorithm

Available on notes.arpanadhikari7.com.np

Step 3: Weight training

Update the weights of the perceptron

$$w(p+1) = w(p) + \Delta w(p)$$

where $\Delta w_i(p)$ is the weight correction at iteration p.

The weight correction is computed by the delta rule:

$$\Delta w_i(p) = \alpha \cdot x_i(p) \cdot e(p)$$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Step 4: Iteration

Increase iteration p by one, go back to *Step 2* and repeat the process until convergence.

Example of perceptron learning: the logical operation AND

Epoch-1 Inputs	$x_1 x_2$ Desired	output-Y_d Initial	weights w_1 w_2	Actual output-Y	Error-e Final	weights w_1 w_2
2 3 4 5	0 1	0 1	0.3 0.2 −0.1	0.1	0 0 1	0	0.3 0.3 −0.1
0 0 1	0 1	0 0 0 1	0.2 0.2 0.0	0.1	0 0 0 1	−1 1	0.2 0.2 0.0
1	0 1	0 0 0 1	0.2 0.1 0.0	0.1	1	0	0.2 0.2 0.0
0 0 1	0 1	0 0 0 1	0.2 0.2 0.0	0.1	0 0 0 1	0 −1 0	0.1 0.2 0.0
1	0 1	0 0 0 1	0.2 0.1 0.0	0.1	0	0	0.2 0.2 0.0
0 0 1	0 1	0 0 0 1	0.1 0.1 0.0	0.1	0 −1 1	0	0.1 0.1 0.0
1	0 1	0.3 0.3	0.1 0.1 0.0	0.1	0	0	0.1 0.1 0.0
0 0 1	0 1	0.3 0.2	−0.1 0.0	0 0 1	0	0.3 0.3	0.1 0.1 0.0
1	0 1	0.3 0.3	−0.1 0.0	0 0 0 1	−1 0	0.2 0.3	−0.1 0.0
0 0 1	0 1		−0.1 0.1	1	0		−0.1 0.1
1							

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1

~~Threshold: θ = 0.2; learning rate: α = 0.1~~

NN Development

development

• There is no single methodology for NN

– A suggested approach is:

Define Problem

Data Selection, Preparation, Preprocessing

Determine Best-Suited Algorithm

Train Network

Trained Successfully?

Test Network

notes.arpanadhikari7.com.np

Tested Successfully?

Run/Use Network

Learning in Neural Nets (again)

- The operation of a neural network is determined by the values of the interconnection weights
  - There is no algorithm that determines how the weights should be assigned in order to solve a specific problems
  - Hence, the weights are determined by a learning process

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Hebbian Learning

- The oldest and most famous of all learning rules is Hebb's postulate of learning:

Available on notes.arpanadhikari7.com.np

“When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic changes take place in one or both cells such that A’s efficiency as one of the cells firing B is increased”

Application to artificial neurons:

If two interconnected neurons are both “on” at the same time, then the weight between them should be increased

notes.arpanadhikari7.com.np

Hebb’s Algorithm

Available on notes.arpanadhikari7.com.np

- **Step 0**: initialize all weights to 0
- **Step 1**: Given a training input, s, with its target output, t, set the activations of the input units: $x_i = s_i$
- **Step 2**: Set the activation of the output unit to the target value: y = t
- **Step 3**: Adjust the weights: $w_i(new) = w_i(old) + x_i y$
- **Step 4**: Adjust the bias (just like the weights): b(new) = b(old) + y

Available on notes.arpanadhikari7.com.np

Example

• **PROBLEM**:
Construct a Hebb Net which performs like an AND function, that is, only when both features are “active” will the data be in the target class.

• **TRAINING SET** (with the bias input always at 1):

```
x1 x2 bias Target 1 1 1 1
1 -1 1 -1
-1 1 1 -1 -1 -1 1 -1
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



Training – First Input

notes.arpanadhikari7.com.np

- Initialize the weights to 0

Present the first input (1 1 1) with a target of 1

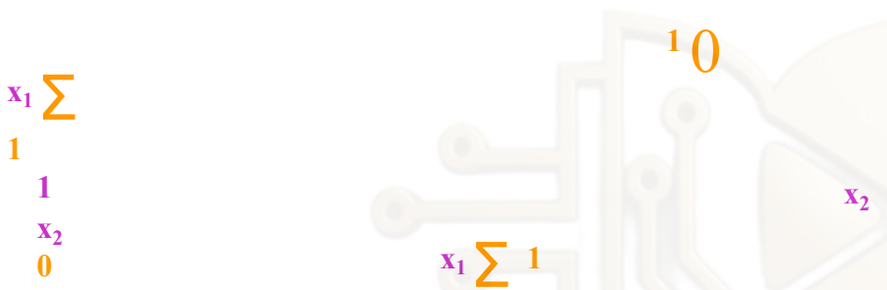
Update the weights:

1

$$b(new) = b(old) + t = 0 + 1 = 1$$

$$w_1(new) = w_1(old) + x_1t = 0 + 1 = 1$$

$$w_2(new) = w_2(old) + x_2t = 0 + 1 = 1$$



Training – Second Input

notes.arpanadhikari7.com.np

- Present the second input:
  (1 -1 1) with a target of -1

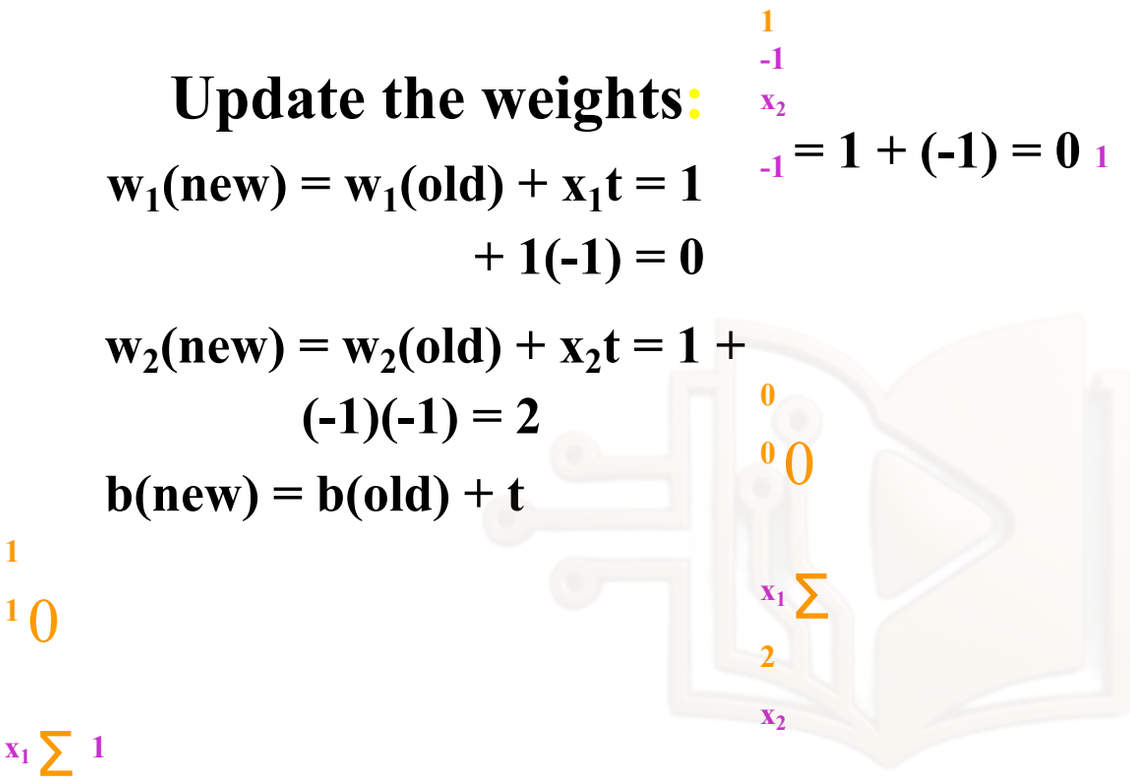
Available on notes.arpanadhikari7.com.np

Update the weights:

$w_1(new) = w_1(old) + x_1 t = 1 + 1(-1) = 0$

$w_2(new) = w_2(old) + x_2 t = 1 + (-1)(-1) = 2$

$b(new) = b(old) + t$



notes.arpanadhikari7.com.np

Training – Third Input

Available on notes.arpanadhikari7.com.np

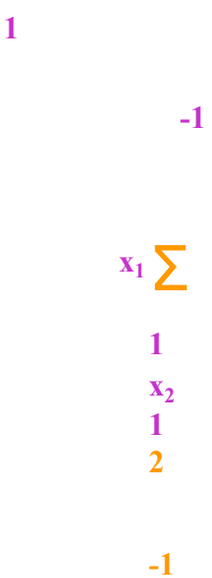
Update the weights:

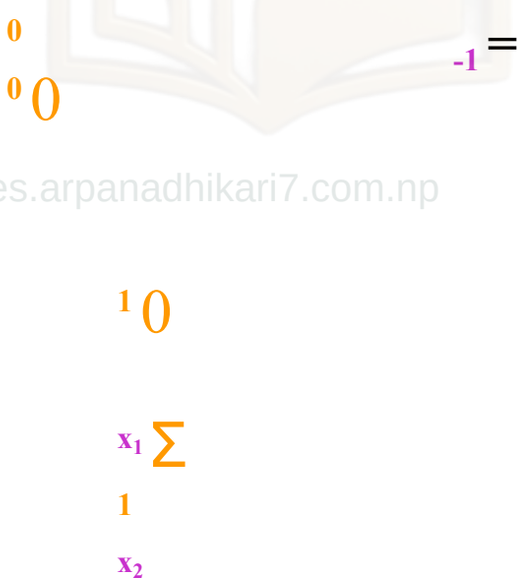
• Present the third input: (-1 1 1) with a target of -1

$w_1(new) = w_1(old) + x_1t = 0 + (-1)(-1) = 1$

$w_2(new) = w_2(old) + x_2t = 2 + 1(-1) = 1$

$b(new) = b(old) + t$





$-1 = 0 + (-1) = -1$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Training – Fourth

Input • Present the fourth

input:
(-1 -1 1) with a target of -1

$w_1(new) = w_1(old) + x_1t = 1 + (-1)(-1) = 2$

notes.arpanadhikari7.com.np

$w_2(new) = w_2(old) + x_2t = 1 + (-1)(-1) = 1$

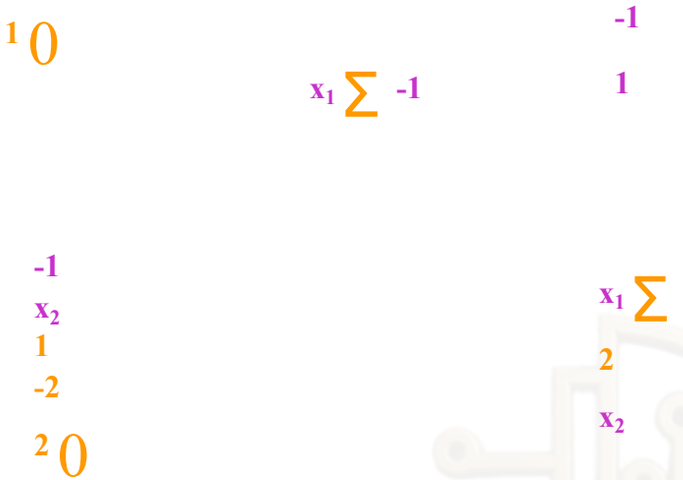
$b(new) = b(old) + t$

$= -1 + (-1) = -2$

1

-1

Update the weights:

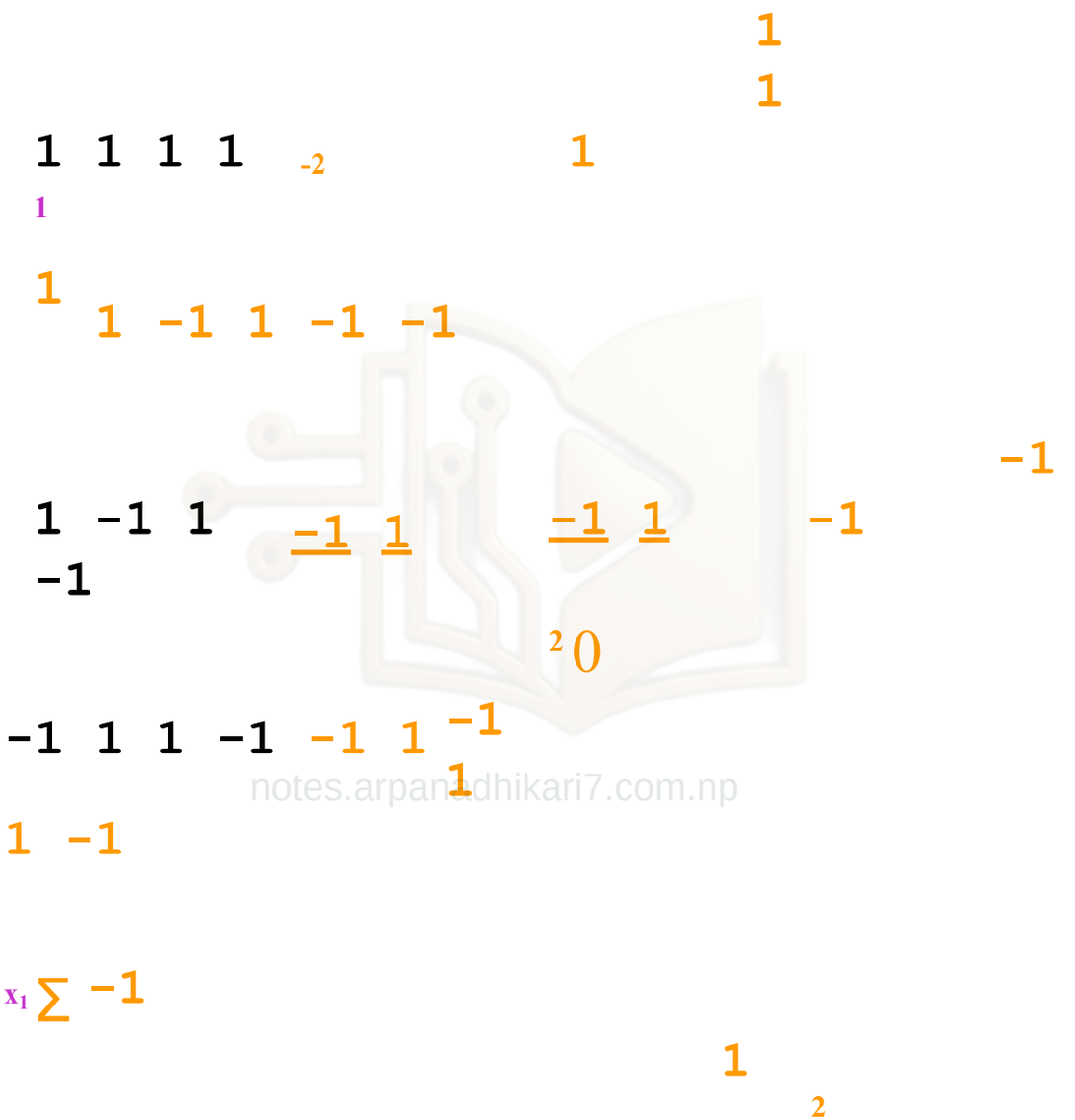


Final Neuron

notes.arpanadhikari7.com.np

- This neuron works:

x1	x2	bias	Target
1	1	1	1



notes.arpanadhikari7.com.np

-1 -1 1 -1
-1 -1 1 -1

x_2

1*2 + 1*2 + 1*(-2) = 2 > 0 (-1)*2
+ 1*2 +1*(-2) = -2 < 0 1*2 +
(-1)*2 + 1*(-2) = -2 < 0 (-1)*2 +
(-1)*2 + 1 *(-2) = -6 <0

OUTLINE

- Introduction to the Perceptron
- Adaline Network

notes.arpanadhikari7.com.np

The Perceptron

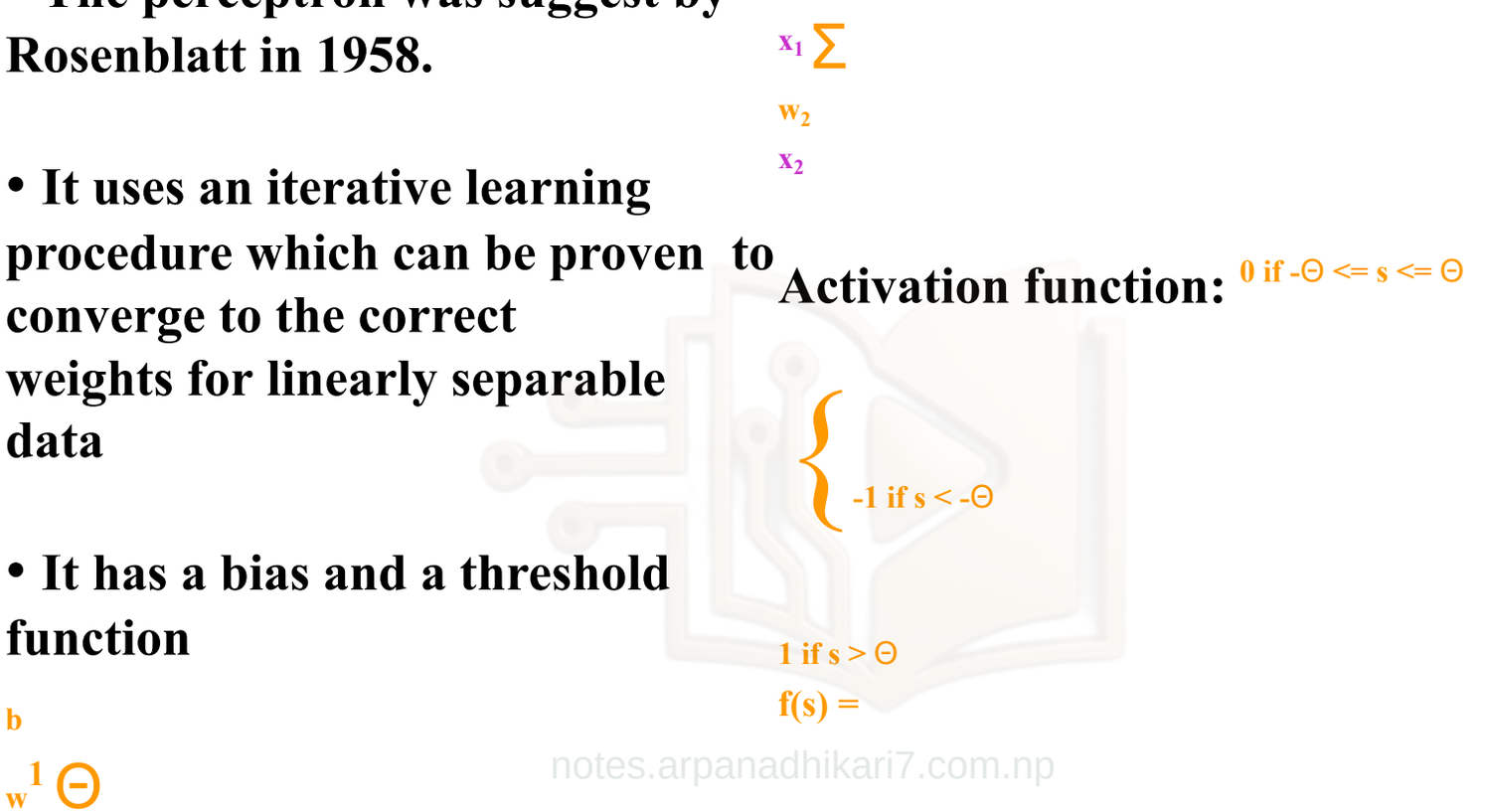
b

Available on notes.arpanadhikari7.com.np

- **The perceptron was suggest by Rosenblatt in 1958.**

- **It uses an iterative learning procedure which can be proven to converge to the correct weights for linearly separable data**

- **It has a bias and a threshold function**



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Perceptron Learning Rule

- Weights are changed only when an error occurs
- The weights are updated using the following:

notes.arpanadhikari7.com.np

$$w_i(new) = w_i(old) + \alpha t x_i$$

t is either +1 or -1
α is the learning rate

If an error does not occur, the weights are

Available on notes.arpanadhikari7.com.np

not changed

Limitations of the Perceptron

The perceptron can only learn to distinguish between classifications if the classes are *linearly separable*.

If the problem is not linearly separable then the behaviour of the algorithm is not guaranteed.

notes.arpanadhikari7.com.np

If the problem is linearly separable, there may be a number of possible solutions.

The algorithm as stated gives no indication of the quality of the solution found.

Available on notes.arpanadhikari7.com.np

XOR Problem

- A perceptron network can not implement an XOR function

notes.arpanadhikari7.com.np

$$w\ w\ w$$

$$+\ +\ \leq$$

$o(x_1,x_2)$

w^0 $w_1w_2\,x_0\,x_1\,x_2$

0. 0. 0
0 1 2

$w\ w\ w$

\+ + >

0. 1. 0
0 1 2

$w\ w\ w$

\+ + >

1. 0. 0
0 1 2

INPUTS ARE DIFFERENT – 1

OUT $w\ w\ w$

\+ + ≤

1. 1. 0
0 1 2

$XOR(x_1,x_2)$

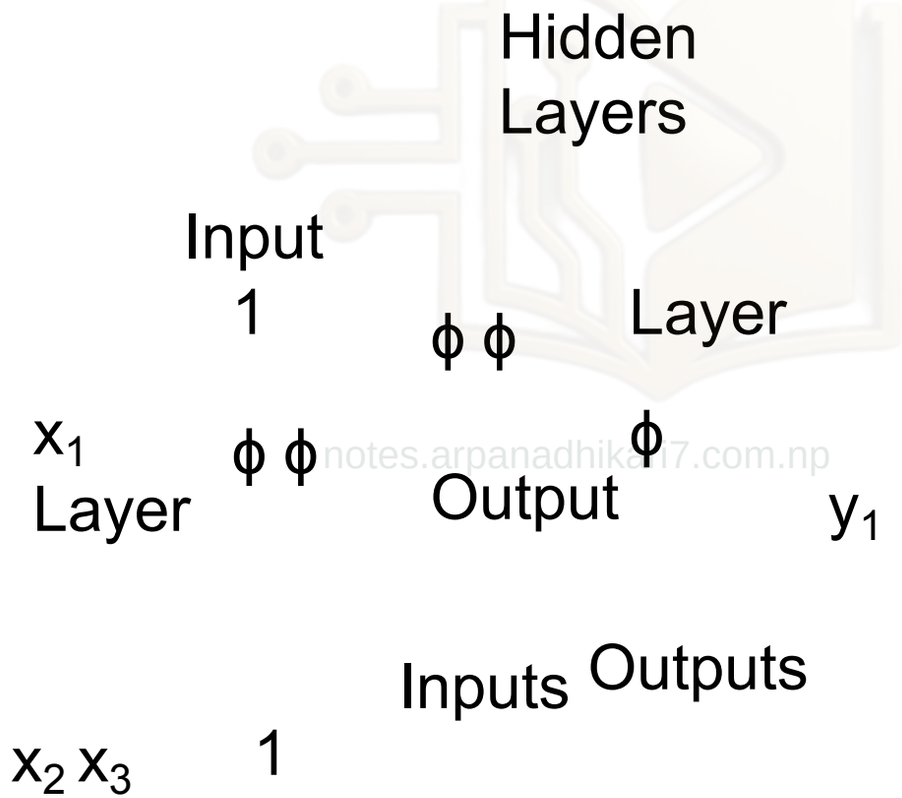
notes.arpanadhikari7.com.np

There is no assignment of values to w_0,w_1 and w_2 that ~~satisfies above inequalities. XOR cannot be represented!~~

Available on notes.arpanadhikari7.com.np

MultiLayer Perceptrons

- Perceptrons can be improved if placed in a multilayered network



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



Adaline Network

notes.arpanadhikari7.com.np

- Variation on the Perceptron Network
  - inputs are +1 or -1
  - outputs are +1 or -1
  - uses a bias input

Available on notes.arpanadhikari7.com.np

- Differences
  - trained using the Delta Rule which is also known as the least mean squares (LMS) or Widrow-Hoff rule
  - the activation function, during training is the identity function
  - after training the activation is a threshold function

notes.arpanadhikari7.com.np

Adaline Algorithm

- **Step 0: initialize the weights to small random values and select a learning rate, α**

Available on notes.arpanadhikari7.com.np

- **Step 1: for each input vector s, with target output, t set the inputs to s**
- **Step 2: compute the neuron inputs**
- **Step 3: use the delta rule to update the bias and weights • Step 4: stop if the largest weight change across all the training samples is less than a specified tolerance, otherwise cycle through the training set again**

Neuron input $y_in = b + x\sum_i w_i$

Delta rule

$b(new) = b(old) + \alpha(t - y_in)$

$w_i(new) = w_i(old) + \alpha(t - y_in)x_i$

notes.arpanadhikari7.com.np

The Learning Rate, α

- The performance of an ADALINE neuron depends heavily on the

Available on notes.arpanadhikari7.com.np

choice of the learning rate

- if it is too large the system will not converge
- if it is too small the convergence will take to long

- Typically, α is selected by trial and error
  - typical range: $0.01 < \alpha < 10.0$
  - often start at 0.1
  - sometimes it is suggested that:

    $$0.1 < n\alpha < 1.0$$

    where n is the number of inputs

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Running Adaline

- One unique feature of ADALINE is that its activation function is different for training and running

- When running ADALINE use the following: –
  initialize the weights to those found during training –
  compute the net input
  – apply the activation function

Neuron input Activation Function

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

$$y_in = b + x\sum^{i} w_i \quad y = \begin{cases} 1 \text{ if } y_in >= 0 \\ -1 \text{ if } y_in < 0 \end{cases}$$

Example – AND function

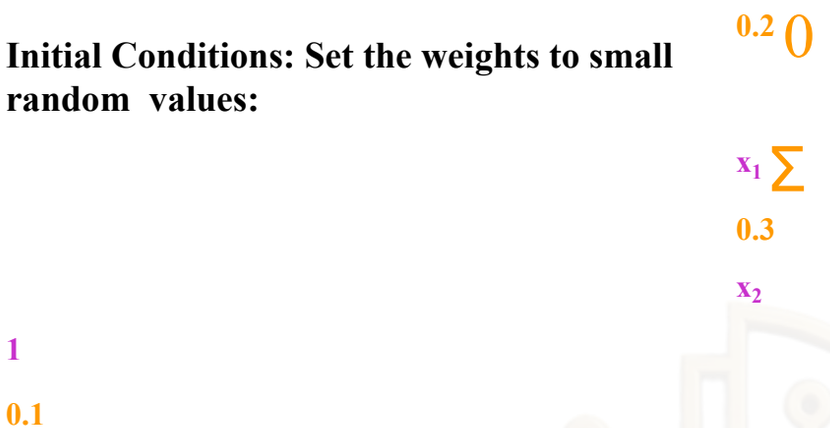
- Construct an AND function for a ADALINE neuron –
  let $\alpha = 0.1$

notes.arpanadhikari7.com.np




```
                    -1 1 1 -1 -1 -1 1 -1
x1 x2 bias Target 1 1 1
1 1 -1 1 -1
```

Initial Conditions: Set the weights to small random values:



First Training Run

notes.arpanadhikari7.com.np

- Apply the input (1,1) with output 1

The net input is:

Available on notes.arpanadhikari7.com.np

$y_in = 0.1 + 0.2*1 + 0.3*1 = 0.6$

1

0.1

0.2 0

1 ∑

1

0.3

1

$b = 0.1 + 0.1(1-0.6) = 0.14$ $w_1 = 0.2 + 0.1(1-0.6)1 = 0.24$ $w_2 = 0.3 + 0.1(1-0.6)1 = 0.34$

The largest weight change is 0.04

The new weights are:

Neuron input $y_in = b + x\sum_i w_i$

Delta rule $w_i(new) = w_i(old) + \alpha(t - y_in)x_i$

$b(new) = b(old) + \alpha(t - y_in)$

notes.arpanadhikari7.com.np

Second Training Run

Available on notes.arpanadhikari7.com.np

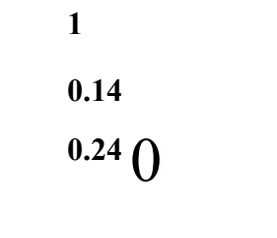
- Apply the second training set (1 -1) with output -1

The net input is:

$$y_in = 0.14 + 0.24*1 + 0.34*(-1) = 0.04$$

The new weights are:

$w_1 = 0.24 - 0.1(1+0.04)1 = 0.14$



$b = 0.14 - 0.1(1+0.04) = 0.04$

The largest weight change is 0.1

0.44 -1

$w_2 = 0.34 + 0.1(1+0.04)1 =$

Available on notes.arpanadhikari7.com.np

Third Training Run

- Apply the third training set (-1 1) with output -1

notes.arpanadhikari7.com.np

The net input is:

$$y_in = 0.04 - 0.14*1 + 0.44*1 = 0.34$$



The new weights are:

b = 0.04 - 0.1(1+0.34) = -0.09

Available on notes.arpanadhikari7.com.np

$w_1 = 0.14 + 0.1(1+0.34)1$
$= 0.27$
-1

0.31

The largest weight change is 0.13

$w_2 = 0.44 - 0.1(1+0.34)1 =$

Fourth Training Run

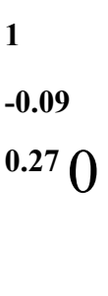
notes.arpanadhikari7.com.np

- Apply the fourth training set (-1 -1) with output -1

Available on notes.arpanadhikari7.com.np

The net input is:

$$y_in = -0.09 - 0.27*1 - 0.31*1 = -0.67$$





The new weights are:

$w_1 = 0.27 + 0.1(1+0.67)1 = 0.43$

$w_2 = 0.31 + 0.1(1+0.67)1 = 0.47$

b = -0.09 - 0.1(1+0.67) = -0.27

-1

The largest weight change is 0.16

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Conclusion

- **Continue to cycle through the four training inputs until the largest change in the weights over a complete cycle is less than some small number (say 0.01)**

- **In this case, the solution becomes: $b = -0.5$, $w_1 = 0.5$, $w_2 = 0.5$**

Example – Recognize Digits

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Multi-Layered Net (again)

notes.arpanadhikari7.com.np

neurons

interconnects

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

output input

notes.arpanadhikari7.com.np

hidden layer

Available on notes.arpanadhikari7.com.np

Properties

- Patterns of activation are presented at the inputs and the resulting activation of the outputs is computed.

- The values of the weights determine the function computed. A network with one hidden layer is sufficient to represent **every Boolean function**. With real weights **every real valued function** can be approximated with a single hidden layer.

Available on notes.arpanadhikari7.com.np

Training

notes.arpanadhikari7.com.np

- Given Training Data,
  - input vector set :

$$\{ \boldsymbol{i}_n | \ 1 < n < N \}$$

  - corresponding output (target) vector set:

$$\{ \boldsymbol{t}_n | \ 1 < n < N \}$$

- Find the weights of the interconnects using training data to ***minimize***

Available on notes.arpanadhikari7.com.np

the error in the test data

Error

input is $\boldsymbol{i}_n$ • Error



notes.arpanadhikari7.com.np

• Input, target & response

– input vector set : $\{ \boldsymbol{i}_n | \ 1 < n < N \}$

– target vector set: $\{ \boldsymbol{t}_n | \ 1 < n < N \}$

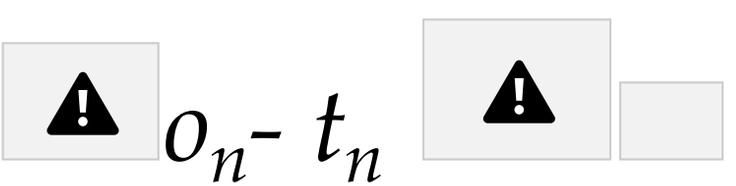
– $\boldsymbol{o}_n$ = neural network output when the

1

2

Available on notes.arpanadhikari7.com.np

notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

Error Minimization Techniques

- The error is a function of the
  - fixed training and test data
  - neural network weights

- Find weights that minimize error (Standard Optimization) –
  conjugate gradient descent
  - random search
  - genetic algorithms
  - steepest descent (error backpropagation)

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Backpropagation

• **Backpropagation is the most well know and widely used neural network system •**

It is a multi-layered, feedfoward, perceptron-like structure • Uses the

backpropagation rule (or generalized delta rule) for training

Available on notes.arpanadhikari7.com.np

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

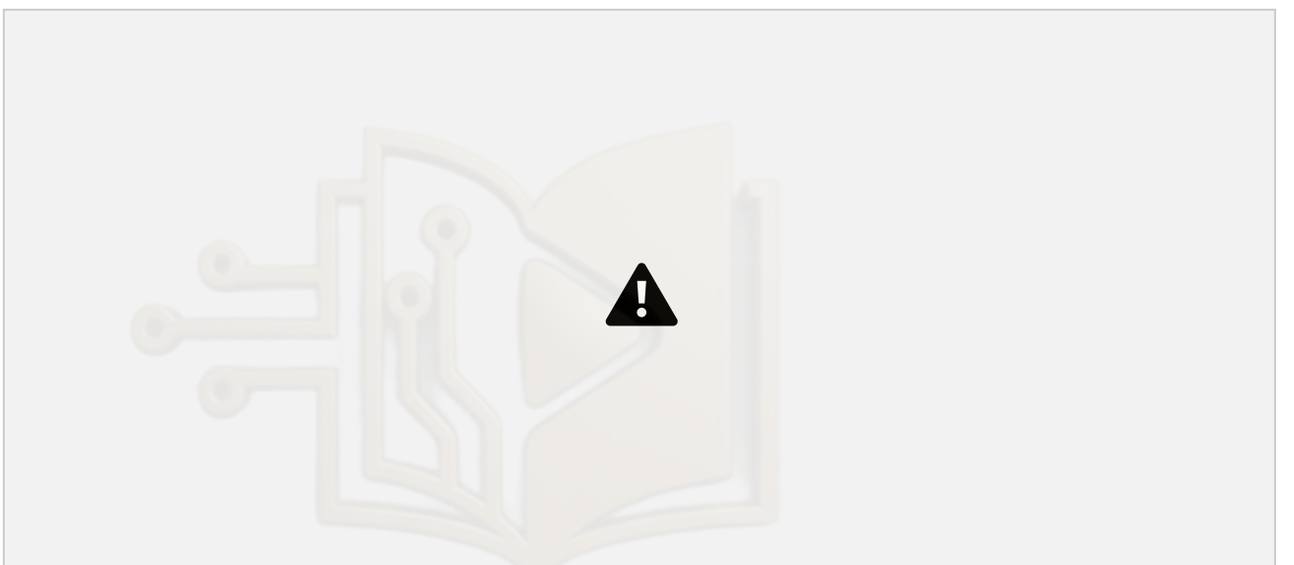


Backpropagation

- Feedforward (for training and normal operation)
- Backpropagation (for error signals)

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Backpropagation Algorithm Training 1

- **Step 0: Initialize the weights to small random values**

Available on notes.arpanadhikari7.com.np

• **Step 1: Feed the training sample through the network and determine the final output** •

Step 2: Compute the error for each output unit, for unit k it is:

$$\delta_k = (t_k - y_k) f'(y_in_k)$$

Actual output

Derivative of f

Required output

notes.arpanadhikari7.com.np

Backpropagation Algorithm Training 2

Available on notes.arpanadhikari7.com.np

- Step 3: Propagate the delta terms (errors) back through the weights of the hidden units where the delta input for the jth hidden unit is:

$$\delta_in_j = \sum_{k=1}^{m} \delta_k w_{jk}$$

notes.arpanadhikari7.com.np

The delta term for the jth hidden unit is:

$$\delta_j = \delta_in_j f'(z_in_j)$$

Available on notes.arpanadhikari7.com.np

Backpropagation Algorithm Training 3

- Step 4: Calculate the weight correction term for each output unit, for unit k it is:

Hidden layer signal

$$\Delta w_{jk} = \alpha \delta_k z_j$$

A small constant

Available on notes.arpanadhikari7.com.np

- Step 5: Calculate the weight correction term for the hidden units:

$$\Delta w_{ij} = \alpha \delta_j x_i$$

Backpropagation Algorithm Training 4

notes.arpanadhikari7.com.np

- Step 6: Update the weights:
-

$$w_{ij}(new) = w_{ij}(old) + \Delta w_{ij}$$

Available on notes.arpanadhikari7.com.np

$$w_{jk}(new) = w_{jk}(old) + \Delta w_{jk}$$

- Step 7: Test for stopping (maximum cylces, small changes, etc)

notes.arpanadhikari7.com.np

Options

- There are a number of options in the design of a backprop system
  - Initial weights – best to set the initial weights (and all other free parameters) to random

Available on notes.arpanadhikari7.com.np

numbers inside a small range of values (say –0.5 to 0.5)

– Number of cycles – tend to be quite large for backprop systems

– Number of neurons in the hidden layer – as few as possible

notes.arpanadhikari7.com.np

Example

Available on notes.arpanadhikari7.com.np

• The XOR function could not be solved by a single layer perceptron network •

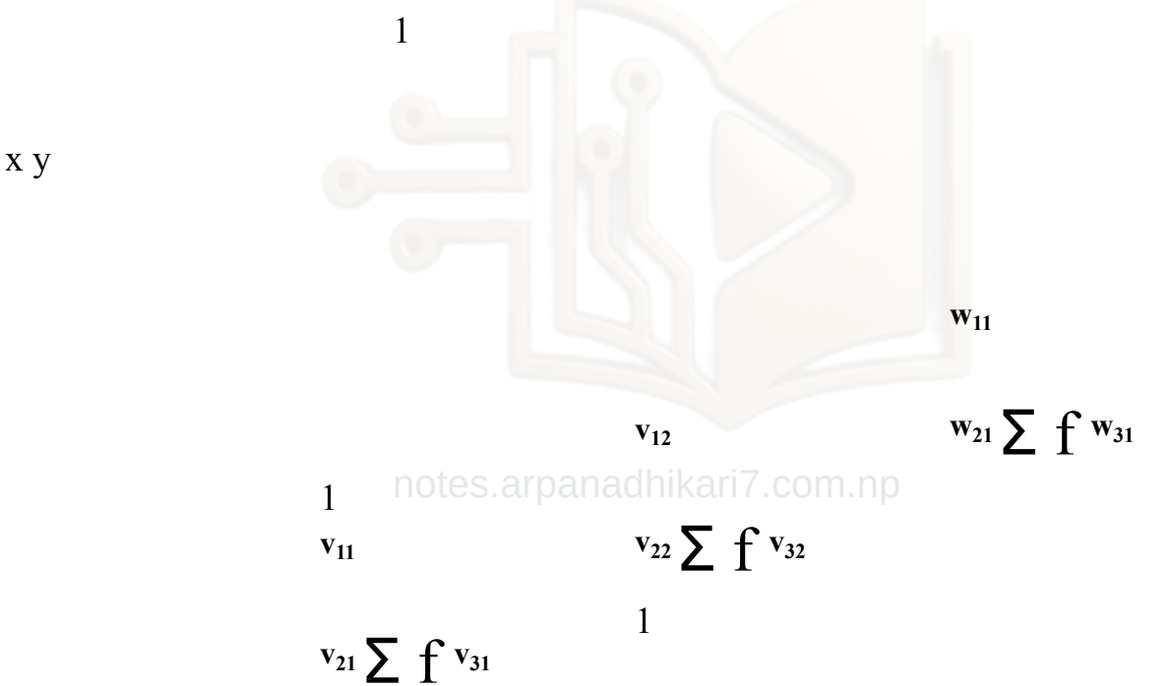
The function is:

X	Y	F
0	0	0
0	1	1
1	0	1
1	1	0

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

XOR Architecture



notes.arpanadhikari7.com.np

Initial Weights

- Randomly assign small weight values:

notes.arpanadhikari7.com.np

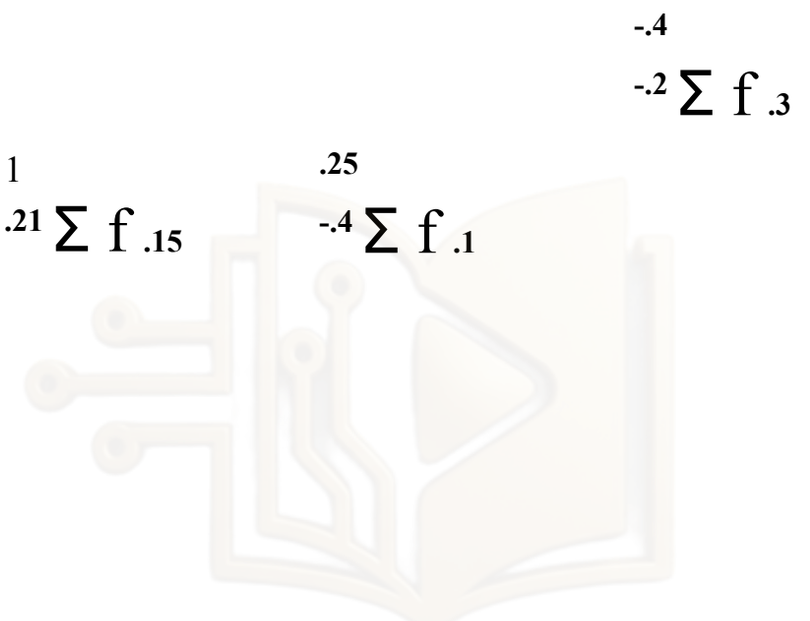
1

1

-.3

Available on notes.arpanadhikari7.com.np

x y



notes.arpanadhikari7.com.np

Feedfoward – 1st Pass

Available on notes.arpanadhikari7.com.np

1

1

x
0

s_1 = -.3(1) + .21(0) + .25(0) =
-.3 f = .43

1
1
-.3
.21 ∑ f
.15

s_3 = -.4(1) - .2(.43) +.3(.56) =

-.318

-.4

1
1

y
0
-.2 ∑ f
.3
s_2 = .25(1) -.4(0) + .1(0)

.25
-.4 ∑ f

f = .42
(not 0)

.1

Training Case: (0 0 0)
f = .56
Activation function f: 1

$y_j = \frac{s_j}{1 + e}$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Backpropagate

$\delta_3 = (t_3 - y_3)f'(y_in_3)$

$=(t_3 - y_3)f(y_in_3)[1 - f(y_in_3)]$

1

0

1

0

-.3

.21 Σ f **.15**

1

-.4 Σ f

.25

-.4

-.2 Σ f **.3**

.1

δ_in₂ = δ₃w₁₂ = -.102(.3) = -.03

$\delta_3 = (0 -$

δ_in₁ = δ₃w₁₃ = -.102(-.2) = .02

δ₁ = δ_in₁f'(z_in₁) = .02(.43)(1-.43) = .005

notes.arpanadhikari7.com.np

$.42).42[1-.42] = \quad -.102$

$\delta_2 = \delta_in_2 f'(z_in_2) = -.03(.56)(1-.56) = -.007$

Update the Weights – First Pass $\delta_3 = (t_3 - y_3) f'(y_in_3)$

notes.arpanadhikari7.com.np

1

$.21 \sum f .15$

0

-.3

Available on notes.arpanadhikari7.com.np

$=(t_3 - y_3)f(y_in_3)[1-$

$f(y_in_3)]$ 1

1

0

$\delta_in_1 = \delta_3 w_{13} = -.102(-.2) = .02$

$\delta_3 = (0 - .42).42[1-.42] = -.102$

$\delta_1 = \delta_in_1 f'(z_in_1) = .02(.43)(1-.43) = .005$

.25
-.4
Σ f .3
-.2

Σ f
-.4
.1

$\delta_in_2 = \delta_3 w_{12} = -.102(.3) = -.03$

~~$\delta_2 = \delta_in_2 f'(z_in_2) = -.03(.56)(1-.56)$~~

$= -.007$

Final Result

Available on notes.arpanadhikari7.com.np

- After about 500 iterations:



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

-.5

$^{1}\sum f_1$

-.5

$^{-2}\sum f_1$

notes.arpanadhikari7.com.np

ppcaons Intrusion Detection ~~Systems~~

- **Anomaly Detection -** The identification of unauthorized system usage

Available on notes.arpanadhikari7.com.np

by discovering statistical variances from established norm, (e.g., profiles of authorized users). Primarily used for detecting internal intrusions.

• **Misuse Detection** - The detection of external attacks ("hackers", etc.). Traditionally addressed by matching current activities to established attacks patterns.

notes.arpanadhikari7.com.np

Sentinel

Available on notes.arpanadhikari7.com.np

- GOAL: Initial design and development of a neural network-based misuse detection system which explores the use of this technology in the identification of instances of external attacks on a computer network.



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Thank you !!!

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np